Reinforcement Learning-Based Path Planning

Riddhima Singh

June 2026

Contents

1 What is RL

Reinforcement learning is learning what to do—how to map situations to actions—so as to maximize a numerical reward signal. The learner is not told which actions to take, but instead must discover which actions yield the most reward by trying them. In the most interesting and challenging cases, actions may affect not only the immediate reward but also the next situation and, through that, all subsequent rewards. These two characteristics—trial-and-error search and delayed reward—are the two most important distinguishing features of reinforcement learning.

Reinforcement learning is different from supervised learning, the kind of learning studied in most current research in the field of machine learning. Supervised learning is learning from a training set of labelled examples provided by a knowledgeable external supervisor. Each example is a description of a situation together with a specification — the label — of the correct action the system should take to that situation, which is often to identify a category to which the situation belongs. The object of this kind of learning is for the system to extrapolate, or generalize, its responses so that it acts correctly in situations not present in the training set. This is an important kind of learning, but alone it is not adequate for learning from interaction. In uncharted territory—where one would expect learning to be most beneficial—an agent must be able to learn from its own experience.

Although one might be tempted to think of reinforcement learning as a kind of unsupervised learning because it does not rely on examples of correct behavior, reinforcement learning is trying to maximize a reward signal instead of trying to find hidden structure. Uncovering structure in an agent's experience can certainly be useful in reinforcement learning, but by itself does not address the reinforcement learning problem of maximizing a reward signal.

One of the challenges that arise in reinforcement learning, and not in other kinds of learning, is the trade-off between exploration and exploitation. To obtain a lot of reward, a reinforcement learning agent must prefer actions that it has tried in the past and found to be effective in producing reward. But to discover such actions, it has to try actions that it has not selected before. The agent has to exploit what it has already experienced in order to obtain reward, but it also has to explore in order to make better action selections in the future. The dilemma is that neither exploration nor exploitation can be pursued exclusively without failing at the task. The agent must try a variety of actions and progressively favor those that appear to be best. On a stochastic task, each action must be tried many times to gain a reliable estimate of its expected reward. The exploration–exploitation dilemma has been intensively studied by mathematicians for many decades, yet remains unresolved.

Beyond the agent and the environment, we can identify four main sub-elements of a reinforcement learning system: a policy, a reward signal, a value function, and, optionally, a model of the environment. A policy defines the learning agent's way of behaving at a given time. Roughly speaking, a policy is a mapping from perceived states of the environment to actions to be taken when in those states.

2 Bandit Problem

The Multi-Armed Bandit problem is a thought experiment in Reinforcement Learning (RL) that illustrates the fundamental trade-off between exploration and exploitation. The situation goes something like: The player walks into a casino and is faced with a row of different slot machines (historically called "one-armed bandits"). They have a limited number of tokens, and each machine has a different, hidden probability of paying out a reward. The goal of the problem is to maximise the total rewards earned by the player, and there are multiple strategies the player could use to go ahead. Each slot machine has a different mean and standard deviation which is unknown to the player.

2.1 Greedy algorithm

The simplest action selection rule is to select one of the actions with the highest estimated value. The player looks at the current knowledge he has and selects the lever which has given him the highest average reward, and then updates the average according to the reward received, choosing the next lever according to the new averages. Greedy algorithms are fast and simple, but they are short-sighted. Because they don't look ahead

to the big picture, they can easily get trapped in a suboptimal solution. A completely greedy algorithm trades off exploration completely for exploitation.

2.2 ϵ-greedy

Here exploitation and exploration are balanced by choosing an ϵ and using a random variable between 0 and 1. If the variable gives a value less than ϵ, then a lever is chosen randomly to pull and the rewards are added and updated in the list of averages. If the variable is greater than our given ϵ then the lever is chosen greedily.

2.3 Optimistic Initial Values

Another interesting approach to dealing with the exploration-exploitation dilemma is to treat actions that we haven't sufficiently explored as if they were the best possible actions. This class of strategies is known as optimism in the face of uncertainty. The optimistic initialization strategy is an instance of this class. The mechanics of the optimistic initialization strategy are straightforward: we initialize the Q-function to a high value and act greedily using these estimates.

2.4 Upper Confidence Bound

In UCB, we're still optimistic, but it's a more a realistic optimism ; instead of blindly hoping for the best, we look at the uncertainty of value estimates. The more uncertain a Q-value estimate, the more critical it is to explore it. Instead of believing everything is roses from the beginning and arbitrarily setting certainty measure values, we follow the same principles as optimistic initialization while using statistical techniques to calculate the value estimates uncertainty and uses that as a bonus for exploration. This is what the upper confidence bound (UCB) strategy does.

$$A_t \doteq \underset{a}{\operatorname{argmax}} \left[Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right],$$

where $N_t(a)$ denotes the number of times that action (a) has been selected prior to time t and the number c >0 controls the degree of exploration. If $N_t(a) = 0$, then (a) is considered to be a maximizing action.

3 Markov Decision Processes

Most real-world decision-making problems can be expressed as RL environments. A common way to represent decision-making processes in RL is by modelling the problem using a mathematical framework known as Markov decision processes (MDPs). In RL, we assume all environments have an MDP working under the hood. The environment is represented by a set of variables related to the problem. The combination of all the possible values this set of variables can take is referred to as the state space. A state is a specific set of values the variables take at any given time. The Markov property: the probability of moving from one state (s) to another state (s) on two separate occasions, given the same action a, is the same regardless of all previous states or actions encountered before that point.

4 Utilization of RL by drones to learn how to fly

Classic drone path-planning uses algorithms like A* or Dijkstra's: you give the computer a map, and it calculates the shortest route mathematically. This works well in simple, known environments, but breaks down when the environment is large, partly unknown, changing, or when the drone has to balance several goals at once (speed, battery life, safety, signal strength, and so on). These classic methods also get very slow as the map grows ('computationally expensive'), and they cannot really learn from experience.

Reinforcement learning flips the approach. Instead of being told the answer, the drone (called the agent) is placed in an environment and allowed to try actions — move forward, turn left, climb, descend — and after

each action it receives a numeric reward (or penalty). Good moves, like getting closer to the destination, earn points; bad moves, like nearing an obstacle or wasting time, lose points. Over thousands of simulated attempts, the drone gradually discovers a strategy, or policy, that racks up the highest total reward.

This is mathematically formalized as a Markov Decision Process (MDP): at every moment the drone is in some state (its position, nearby obstacles, battery, etc.), it takes an action, and the environment returns a new state and a reward. Crucially, the next state only depends on the current state and action — not on the entire history — which is what makes the problem tractable to solve with algorithms like Q-learning.

Why this matters for drones specifically: a UAV's surroundings (wind, dynamic obstacles, no-fly zones, lost GPS signal, multiple competing mission goals) are messy and unpredictable in ways that are hard to hand-code. RL lets the vehicle build its own internal sense of "what works" from simulated trial and error — usually in a virtual environment first, since crashing a real drone thousands of times would be expensive and dangerous — and then transfer that learned behavior to real flights.

5 Research paper 1 – Q Learning vs SARSA

UAV Path Planning and Obstacle Avoidance Based on Reinforcement Learning in 3D Environments

`https://www.mdpi.com/2076-0825/12/2/57`

5.1 What is Q-Learning

Q-Learning is one of the most famous and foundational algorithms in Reinforcement Learning. It is a model-free, value-based algorithm whose goal is to find the best possible policy (rulebook) for an agent to follow to maximize its total long-term reward. The "Q" stands for Quality. It represents how useful a specific action is in a specific state for gaining future rewards. Model free means that there is no exact simulation of the environment that it can train on or no model example for it to follow. It must explore on its own.

The Q table: When the agent starts, this entire table is filled with zeros because the agent knows nothing about the world. As the agent explores the environment (using strategies like the ϵ-greedy algorithm), it updates the cells in this table. Over time, this table becomes like a cheat sheet telling the agent exactly which action has the highest quality in any given state.

Q-Learning formula: To update the Q-table, the algorithm uses a variation of the famous Bellman Equation. Every time the agent takes an action, it looks at the immediate reward it received, plus the best possible reward it expects to get in the next state, and updates its current knowledge.

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[R(s,a) + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

- $Q(s,a)$ **(Current Q-Value):** What the agent currently thinks the quality of taking action a in state s is.
- α **(Learning Rate):** A number between 0 and 1. It determines to what extent newly acquired information overrides old information. If $\alpha = 0$, the agent learns nothing. If $\alpha = 1$, the agent completely replaces old knowledge with the newest reward.
- $R(s,a)$ **(Reward):** The immediate, actual reward the agent just received for taking action a in state s.
- γ **(Discount Factor):** A number between 0 and 1. It determines the importance of future rewards. A discount factor of 0 makes the agent "myopic" (only caring about immediate rewards), while a factor approaching 1 makes it strive for a long-term high reward.
- $\max_{a'} Q(s',a')$ **(Maximum Expected Future Reward):** The agent looks at the next state (s') it landed in, looks at all possible actions (a') in that next state, and grabs the highest Q-value available.

5.2 What is SARSA

SARSA (State-Action-Reward-State-Action) is an "on-policy" Reinforcement Learning algorithm. It is very similar to Q learning however there is a very slight difference. Q learning assumes that the next action it will take will be the best possible action while updating its Q table, regardless of what its next action actually is (random or best possible) meanwhile SARSA considers what its next action actually will be while updating its own table and deciding what to do next. It takes into account if its next action will be random or not.

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[R + \gamma Q(s',a') - Q(s,a)\right]$$

The a' here is **NOT** the $\max a'$ like in the case of Q Learning.
An example which explains this is:
If there is a cliff and a path that runs right along the edge of it-

- Q-Learning will learn to walk right along the edge, because it assumes it will always take the "optimal" (non-random) action, so it ignores the tiny chance of a random slip.
- SARSA will learn to stay further away from the edge. It knows that its own ϵ-greedy policy might cause it to make a random, bad move. Since walking near the edge means a random move could result in falling off the cliff, SARSA learns that the edge is "expensive" in terms of potential negative rewards.

Hence SARSA operates as a safer way of learning while Q Learning focuses more on optimization despite being more risky.

5.3 The problem statement in the research and outcome

The authors needed a hexacopter (six-rotor drone) to inspect aquaculture net-cages scattered across the open sea, fly safely to each one, drop a sensor, and return to base — all while avoiding obstacles such as masts, sails, and uneven terrain that GPS alone cannot detect. The two sub-problems are: (1) plan an efficient global route around known no-fly zones before take off, and (2) react in real time to obstacles the drone only discovers visually during flight.

In the study, the path planning was treated as a "global path planning" problem, meaning the drone had a prior understanding of the static obstacles (like no-fly zones) before taking off. To solve this, the researchers transformed the environment into a simplified mathematical game for the agent to solve.

Before the drone flew, the working area was mapped out as a two-dimensional grid.

- **State:** The drone's current coordinate on this grid map.
- **Actions:** The drone was limited to four movement options: moving up, down, left, or right.

The game which was set up then had rules as following:

To force the algorithm to find the absolute best path, the researchers set up a strict reward and punishment rulebook:

- **Step Penalty (-1 point):** For every single square the drone moved, it lost 1 point. This taught the agent to reach the goal using the fewest possible steps.
- **Obstacle Penalty (-200 points):** If the drone stepped into a restricted "no-fly zone" (an obstacle), it received a massive penalty of -200 points and was teleported back to the starting point.
- **Goal Reward (+100 points):** Reaching the final destination yielded a large positive reward.

Figure 11. The schematic diagram of the reward and punishment rules.

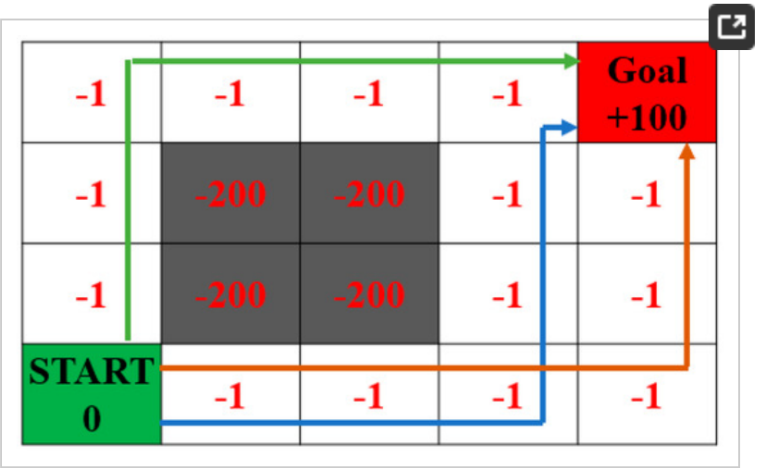

Figure 12. The routes in the reward and punishment rules.

The ultimate objective for the algorithm was to maximize its total score over the entire trip. During training, the algorithms didn't just greedily pick the best path immediately. They used an ϵ-greedy policy.

The difference between using Q Learning and SARSA: The Q-learning algorithm is an off-policy. This different policy means that the action policy and the evaluation policy are not the same. The action policy in Q-learning is the ϵ-greedy policy, whereas the policy to update the Q-table is the greedy policy. On the other hand, with SARSA the on-policy update method is adopted, and both the action policy and the update policy are an ϵ-greedy policy. This means that the SARSA algorithm will directly adopt the policy that is currently considered the best in the current state.

The same difference was observed between the two algorithms SARSA and Q Learning as expected—SARSA picked a path which was further away from the no fly zones while Q Learning picked the actual most optimal path that was possible.

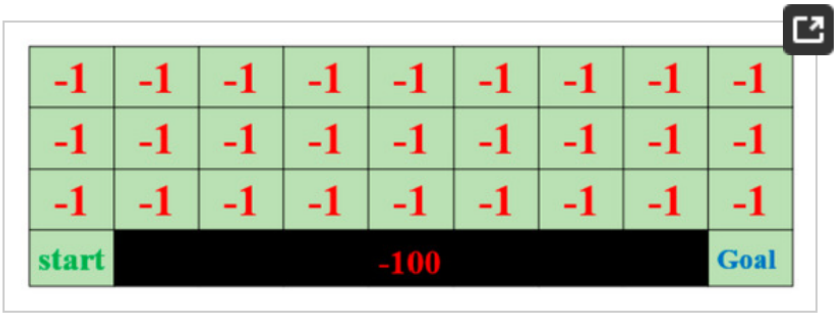

Figure 1: Grid

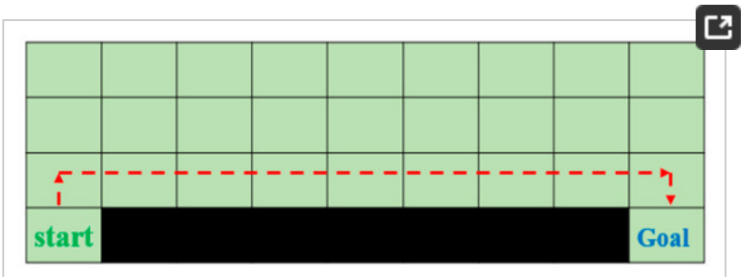

Figure 2: Q-Learning Path

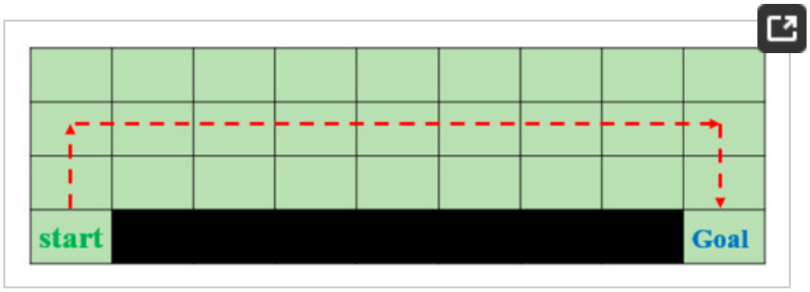

Figure 3: SARSA Path

Although the Q-learning algorithm has the ability to learn the globally optimal path, its convergence is slow, while the SARSA algorithm has a lower learning effect than the Q-learning algorithm, but its convergence is faster and simpler.

Here, the obstacles were somewhat known beforehand. Pathfinding in a known environment is known as global path planning. The main purpose is to spend less time and reach the destination without passing through restricted areas. Compared with the traditional A* algorithm, the reinforcement learning algorithm can solve more complex problems which is why its implemented here. For dynamic obstacle avoidance, the authors of the paper used deep Q-Learning and incorporated neural networks instead of simple Q tables as the number of possible states became massive. The use of deep Q-learning in the virtual environment of Microsoft AirSim requires a great deal of computer resources; therefore, a graphics card with better graphics computing capabilities and sufficient memory, etc. is required.

6 DQN- Google DeepMind's breakthrough

Deep Q Network is like the combination of Q Learning with Deep Neural Networks. In traditional Q-Learning, an agent learns to navigate an environment by maintaining a giant cheat sheet called the Q Table. The rows represent every possible state (situation). The columns represent every possible action. The cells contain a Q-value (Quality value), which estimates the total future reward the agent will get if it takes that action in that state.

This works for small tables, however when the number of possible states and actions are massive, this approach fails as the data is too much to store. This is where it is replaced by Deep Neural Networks. Instead of looking up a value in a table, the agent feeds the current state into the neural network, and the network predicts the Q-values for all possible actions.

The core architecture follows:

- **Input:** The current state s (e.g., the last 4 frames of a video game to capture movement).
- **Output:** A vector of continuous numbers. Each number represents the predicted $Q(s, a)$ for a specific, discrete action (e.g., [Go Left: 0.2, Go Right: 1.5, Jump: 4.2]).
- **Decision:** The agent typically uses an ϵ-greedy (epsilon-greedy) strategy. Most of the time, it picks the action with the highest Q-value (exploitation), but with a small probability ϵ, it picks a random action to discover new strategies (exploration).

6.1 Drawbacks faced and how they were fixed

- In RL, consecutive steps are highly correlated. Training a neural network on highly correlated, sequential data violates the fundamental machine learning assumption that data should be independent and identically distributed. It causes the network to overfit to its immediate past. As a solution, DQN uses a memory buffer. As the agent interacts with the environment, it saves its experiences as tuples. When it comes time to train the network, instead of using the immediate last step, it randomly samples a small batch of random experiences from across its entire history. Due to this, the network does not overfit its data despite consecutive frames captured by a drone being highly correlated.
- Usually in deep learning, there is a fixed target. However, when Q Learning is combined with deep learning, the target also keeps on changing as the network looks for the maximum possible reward which is calculated using the Q Learning as well. This becomes like an endless loop where the input and target keep on updating. To fix this, DeepMind maintained two separate networks- Policy network and target network, where the target network remained mostly fixed while the policy network changed each time. This way the neural network had a fixed target.

7 Research paper 2- Soft Actor-Critic Algorithm

`https://www.mdpi.com/2313-7673/8/6/481`

SAC is one of the most recent break throughs in path planning for UAV systems and drones based on RL. While maximum techniques before divide actions of the drone into discrete parts such as turn left, turn right, move forward etc, SAC outputs a continuous probability distribution as response. It gives the mean and standard deviation and then a value is chosen from this distribution and tried out by the model.

7.1 Some key new concepts

- **Artificial Potential Fields**

  A major hurdle in training UAVs is the sparse rewards where the drone only receives meaningful feedback upon crashing or reaching the target. To accelerate learning, researchers often combine SAC with APFs. The APF provides a dense, continuous reward gradient instead of discrete values as it mathematically pushes the drone away from obstacles (repulsive forces) and pulls it toward the target (attractive forces) at every step.

- **Output of fluid distributions instead of discrete steps**

  A vector is output with the velocity, thrust, etc instead of a discrete value (such as increasing speed by 10%). Since the output is a probability distribution curve, the algorithm randomly samples an actual action from that bell curve. If σ is large, the curve is wide, and the drone might try a wildly different thrust value. If σ is tiny, the curve is narrow, and the drone will choose an action very close to the mean. Early in training, the network is highly uncertain, so it outputs a massive σ. The drone explores all sorts of erratic movements. As it learns what works, the network naturally shrinks σ around the successful maneuvers, transitioning smoothly from frantic exploration to precise control.

This is different from the simple algorithm used before, where the output was only left, right, up and down as manoeuvring a drone involves much more than that. The drone here perceives its surroundings through depth maps generated from camera imagery, which give it a sense of how far away nearby obstacles are in every direction.

SAC also works on the principle of Maximum Entropy Reinforcement Learning, where it focuses on maximizing both the expected reward and the entropy (randomness) of the policy, unlike most models that focus only on the reward. By maximizing entropy, SAC forces the UAV to explore as broadly as possible while still completing the objective. This prevents the drone from prematurely locking onto a suboptimal flight path (local optima) and makes the final policy highly robust to real-world perturbations, such as sudden wind gusts.

7.2 Side note

This focus on making sure that the algorithm is not stuck on a local maxima and explores more paths as well simultaneously is similar to another famous path planning algorithm- Ant Colony Algorithm. This follows the biological method of ants managing to find the shortest root possible to their food.

However, in normal ACA algorithm, once a local maxima is found, it is possible that all the ants continue to follow that path only and not explore for a possibly more optimal path. This is countered by the CACA Algorithm, which stands for Chaotic Ant Colony Algorithm which ensures that there is chaos among the ants so that pathways are discovered and explored much more broadly.

This is combined with RL algorithms to provide initial data on the paths to the model, which are then explored further using the policies stated above.

7.3 MDP for SAC

The MDP here is built differently compared to the previous simpler model. Here, the MDP is built as:

- **State Space (s_t):** Typically includes the drone's 3D coordinates, velocity vectors, attitude and sensor arrays measuring distances to the nearest obstacles.
- **Action Space (a_t):** Continuous vectors, usually representing target velocities or direct rotor thrusts.
- **Reward Function (r_t):** The scalar feedback evaluating the safety and progress of the drone.

These represent the flight of a drone much more realistically, keeping in mind all the environmental factors that the drone might face while flying.

7.4 Actor-Critic Architecture

One neural network (the "actor") decides what action to take, while a second network (the "critic") evaluates how good that action turned out to be, and the two are trained together so the actor keeps improving based on the critic's feedback. It is the heavy focus on entropy which makes the actor output a Gaussian distribution instead of a single, rigid action which is where the term "Soft" comes from.

7.5 Difference between DQN and SAC

- **Action Space:** DQN only works for discrete action spaces (e.g., up, down, button A, button B). It cannot handle continuous control like the precise steering angles or dynamic rotor thrusts that SAC manages, which makes SAC more fit for shifting from simulations to real life flight tests for drones.
- **Architecture:** DQN is a value-based method which only guesses values, and the policy is just picking the highest value, based on ϵ-greedy. SAC is an actor-critic method and has a dedicated network to choose actions and another to evaluate them.